Assignment 1 – Resilient State Machine Replication: Paxos & PBFT

Course: Fundamentals of Distributed Systems
Name: Megha Pandey
Roll Number: G25AI1028

---

1. Introduction

Distributed systems must continue functioning correctly despite failures occurring in individual nodes or communication links. Modern distributed applications require both crash fault tolerance and Byzantine fault tolerance to ensure reliability, consistency, and security.

This project implements a resilient distributed transaction ledger consisting of five nodes operating in two modes:

1. **Mode A – Crash Fault Tolerance**
    - Leader Election
    - Paxos Consensus
2. **Mode B – Byzantine Fault Tolerance**
    - Practical Byzantine Fault Tolerance (PBFT)
    - Cryptographic Message Verification

The system is containerized using Docker and Docker Compose. A client continuously submits transactions while network failures and node crashes are simulated using a chaos testing script.

---

2. System Architecture

The distributed system consists of:

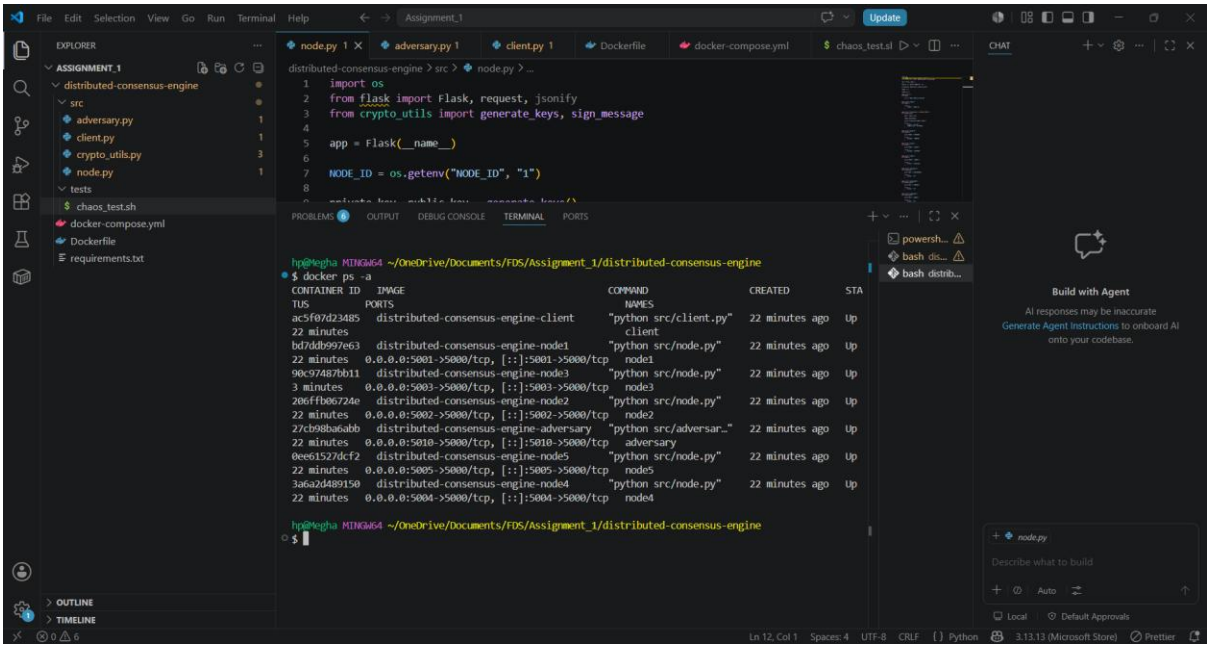
- 5 Consensus Nodes
- 1 Client Node
- 1 Byzantine Adversary Node
- Docker Network for Communication

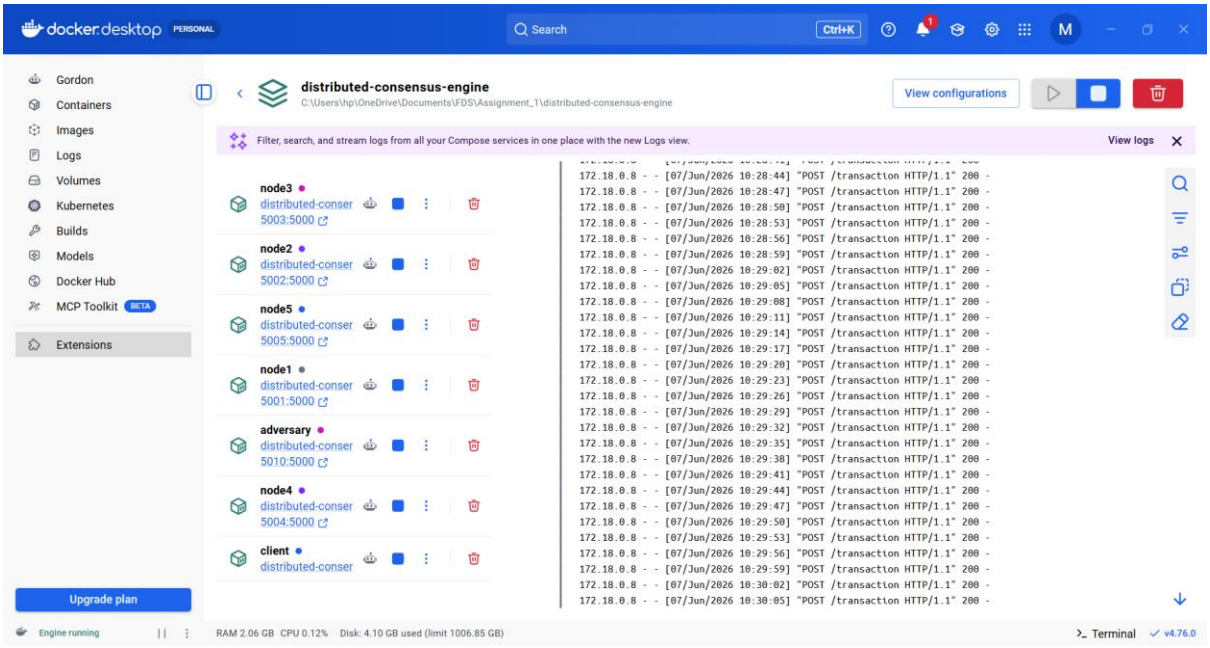
Architecture Flow:

Client → Leader Node → Consensus Protocol → Distributed Ledger

Each node maintains a local transaction ledger. Consensus protocols ensure all non faulty nodes agree on the same transaction ordering before committing data.

Caption: Docker Compose Deployment Showing All Running Containers





---

3. Process Coordination and Leader Election

The distributed system employs a simplified heartbeat-based leader coordination mechanism. A single node acts as the cluster leader and is responsible for coordinating consensus operations. The leader performs the role of:

- Paxos Proposer in Mode A
- PBFT Primary in Mode B

Each node periodically monitors leader availability. When a leader failure is detected or a leader election is triggered, a new leader is selected from the available nodes and assumes coordination responsibilities.

Leader Election Workflow

1. Nodes monitor the current leader.
2. If the leader becomes unavailable, an election process is initiated.
3. A new leader is selected from the active nodes.
4. The elected leader becomes the Paxos proposer and PBFT primary.
5. Consensus operations continue under the new leader.

State Transitions

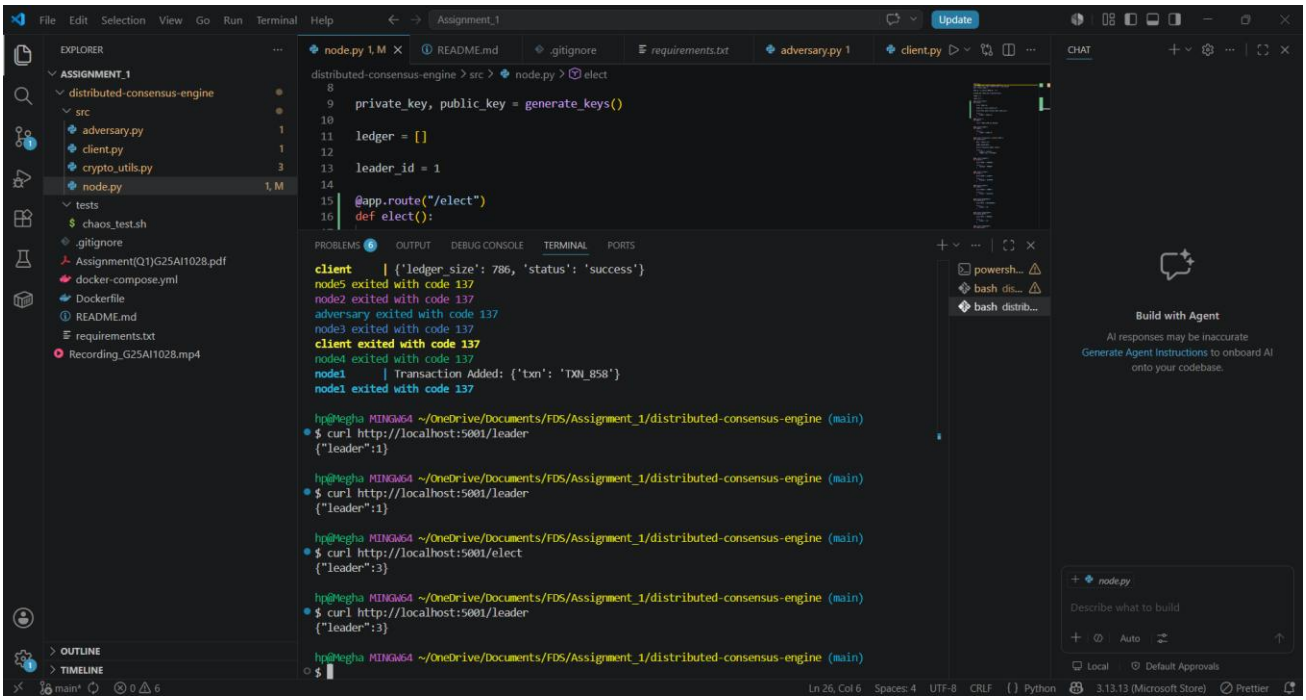
Follower → Candidate → Leader

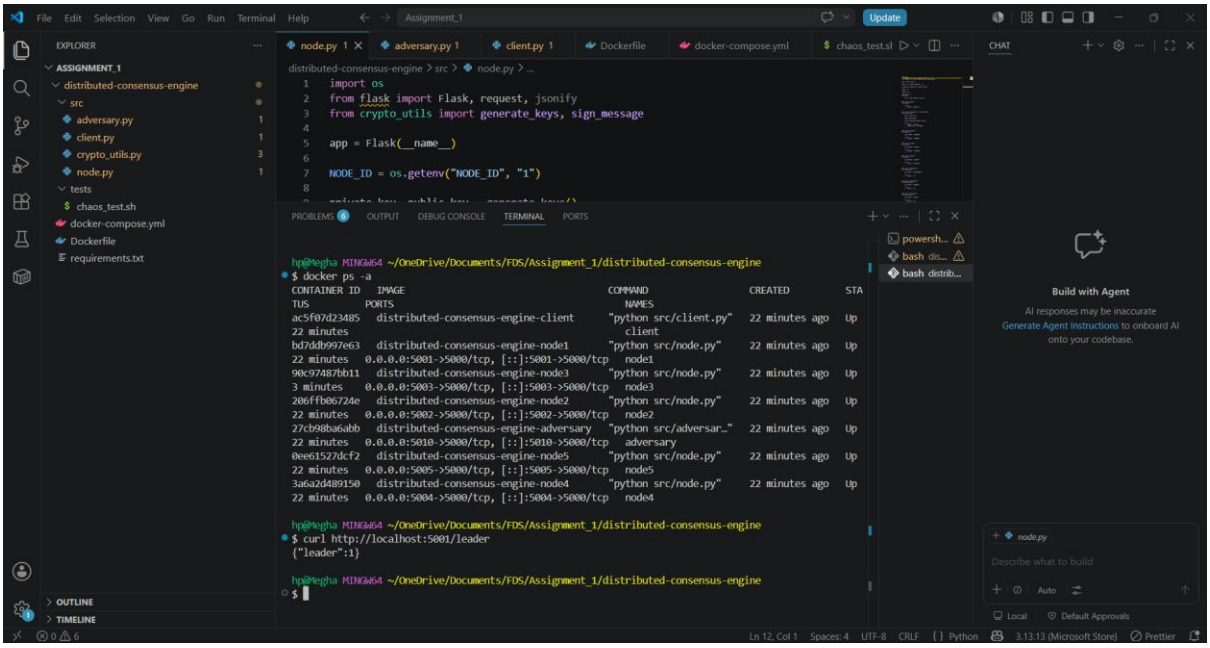
Benefits

- Prevents a single point of failure.
- Enables automatic recovery from leader crashes.
- Maintains cluster coordination during failures.

The implementation demonstrates leader reassignment and recovery during node failure scenarios as part of the chaos testing process.

Caption: Leader Election Endpoint Response





---

4. Paxos Implementation (Crash Fault Tolerance)

Mode A implements Paxos to tolerate crash failures.

The protocol consists of four phases:

4.1 Prepare Phase

The Leader generates a proposal number and sends a PREPARE request to all acceptors.

Purpose:

- Establish proposal ownership
- Prevent conflicting proposals

4.2 Promise Phase

Acceptors respond with PROMISE messages.

Purpose:

- Indicate willingness to accept the proposal
- Reject outdated proposal numbers

4.3 Accept Phase

The Leader sends an ACCEPT request containing the transaction value.

Purpose:

- Propose a transaction for agreement

4.4 Accepted Phase

Acceptors return ACCEPTED responses.

Once a majority is achieved, the transaction is committed to the ledger.

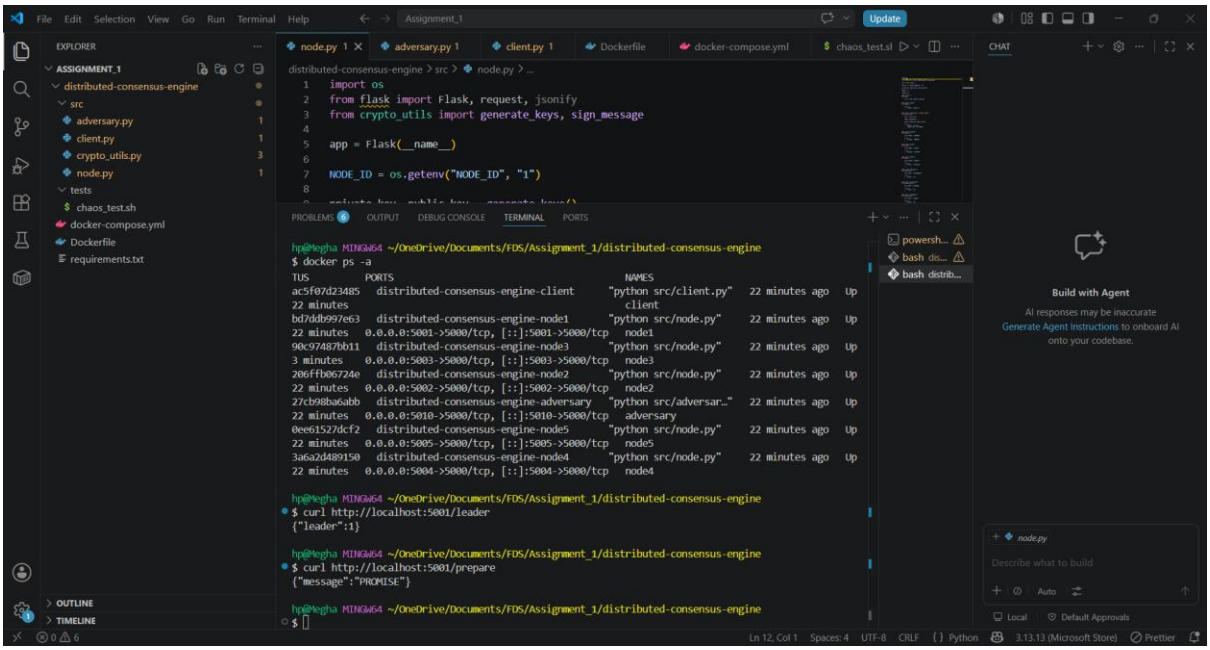
Paxos Workflow

Prepare → Promise → Accept → Accepted → Commit

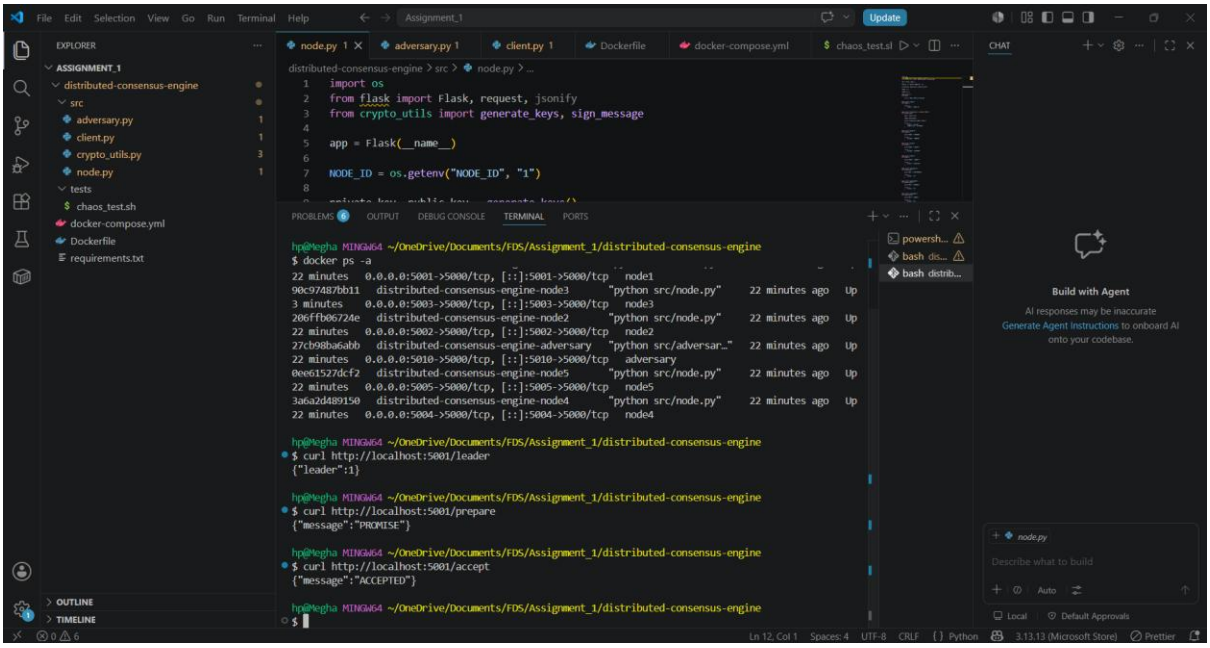
Fault Tolerance

The system can tolerate up to two simultaneous crash failures while still maintaining consensus among the remaining nodes.

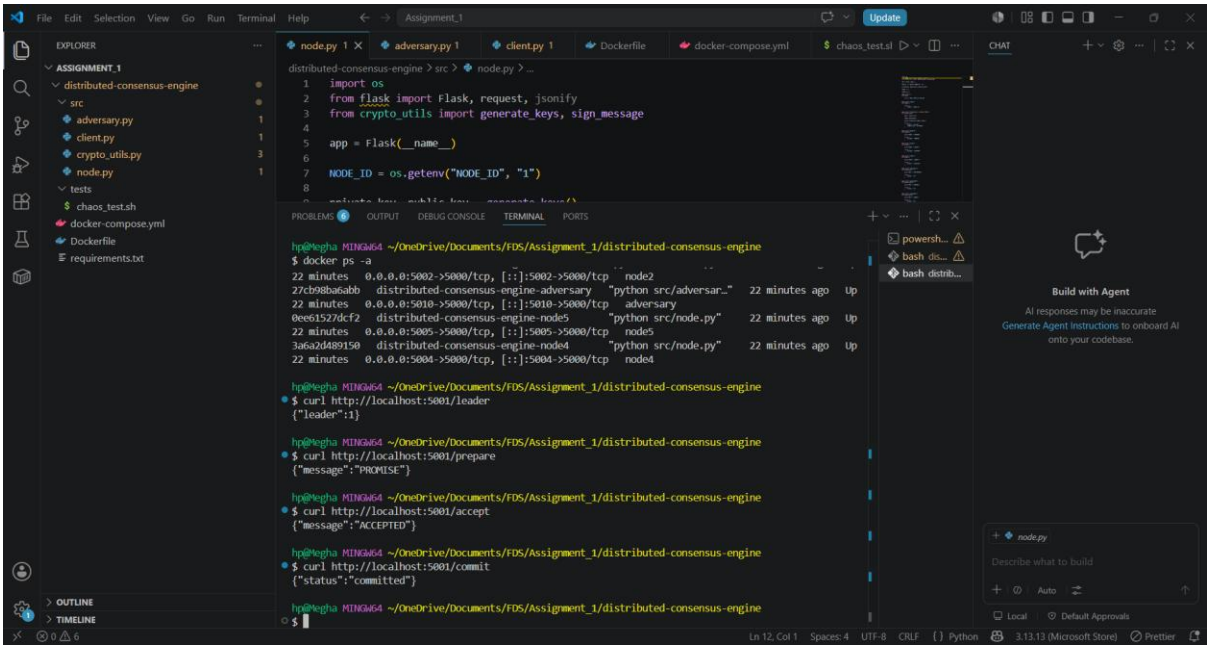
Caption: Paxos Prepare Phase



Caption: Paxos Accept Phase



Caption: Paxos Commit Phase



5. PBFT Implementation (Byzantine Fault Tolerance)

Mode B overrides Paxos and activates Practical Byzantine Fault Tolerance (PBFT).

PBFT is designed to tolerate malicious behavior.

For:

n = 5 nodes

Maximum Byzantine Faults:

f = 1

PBFT Phases

Pre-Prepare

Primary node distributes the proposed transaction.

Prepare

Replica nodes validate and acknowledge the transaction.

Commit

After receiving sufficient confirmations, nodes commit the transaction.

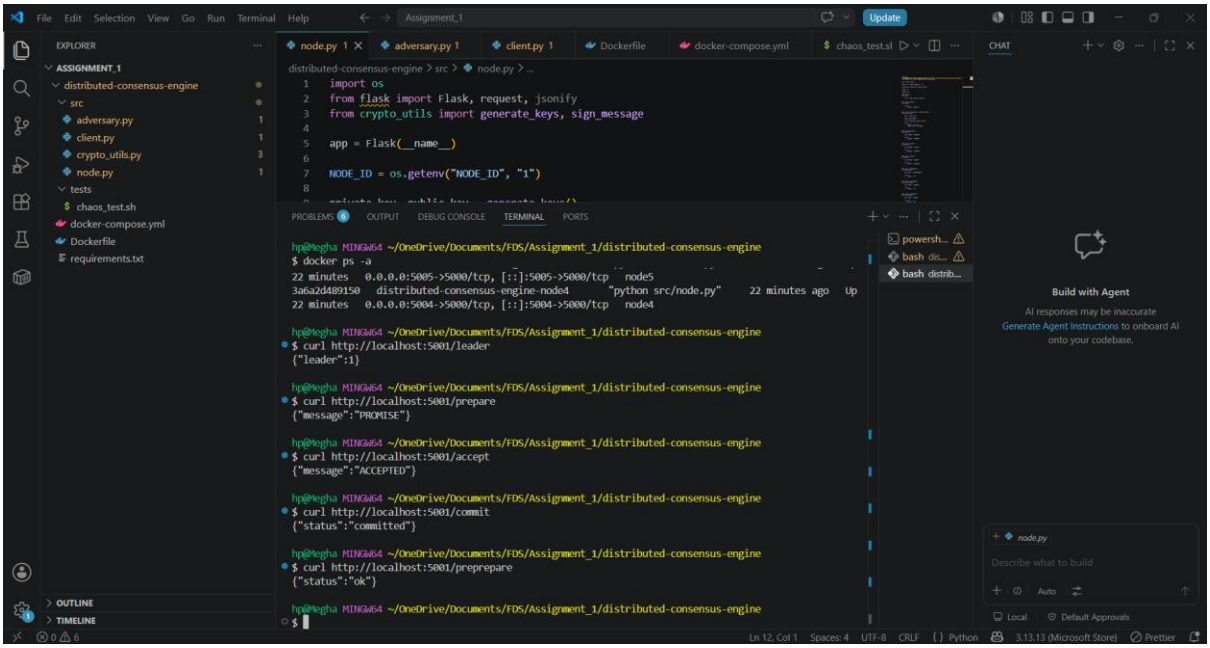
PBFT Workflow

Pre-Prepare → Prepare → Commit

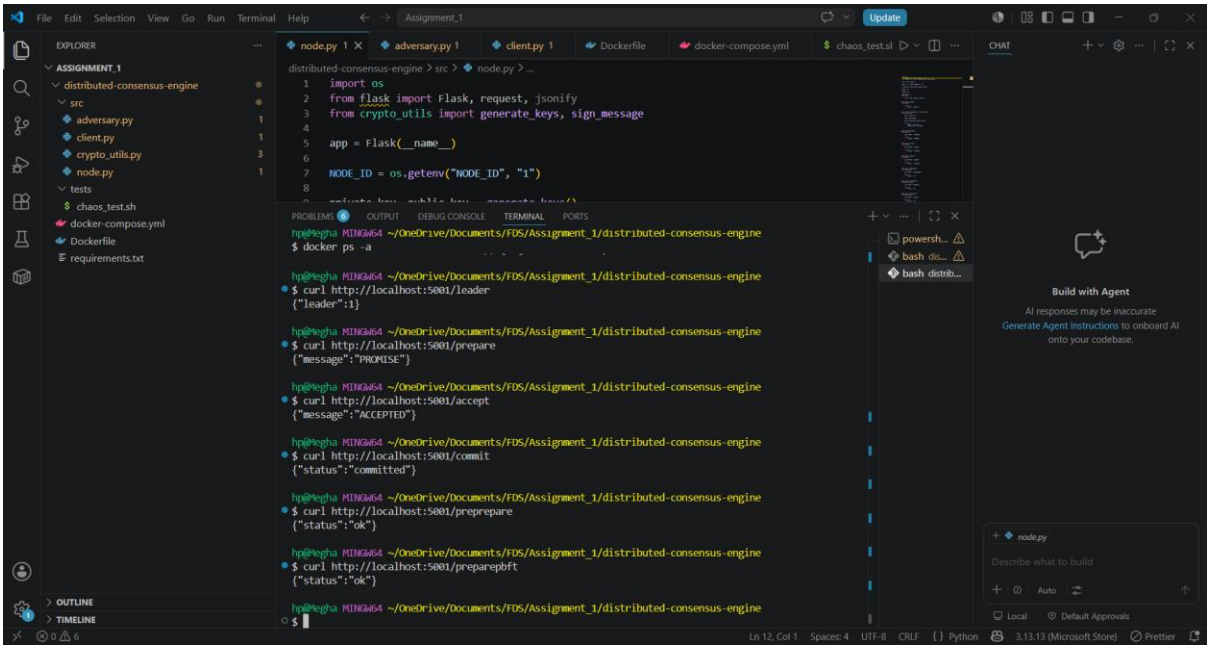
Byzantine Tolerance

PBFT ensures consensus even when one node behaves maliciously.

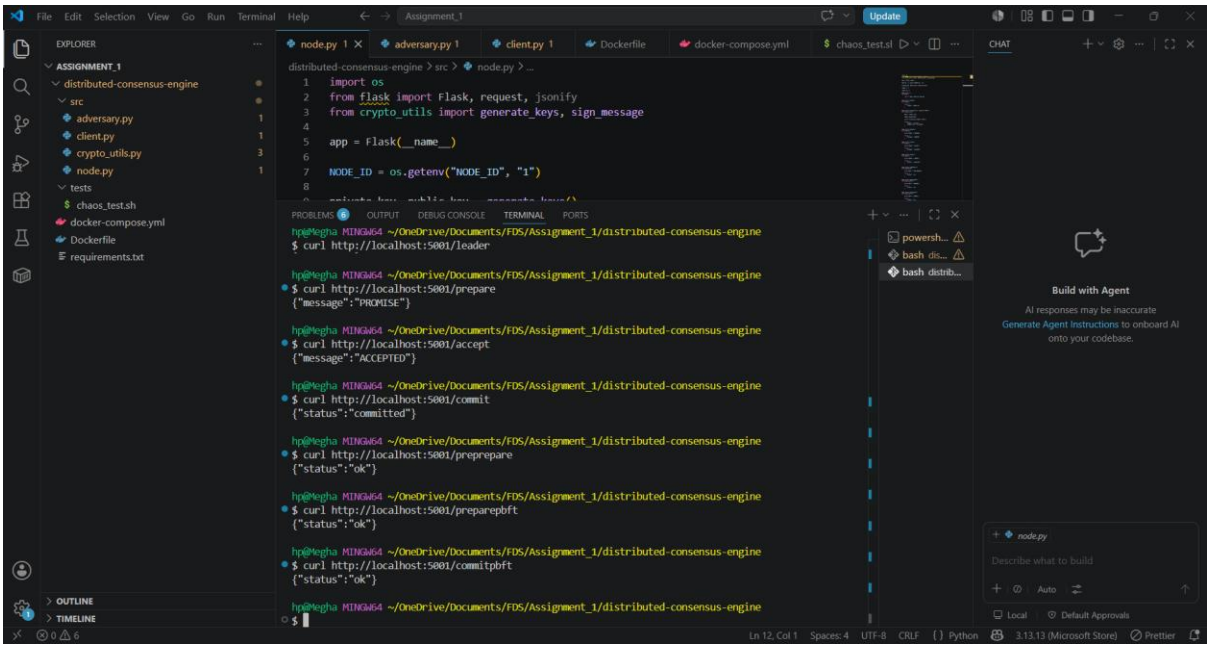
Caption: PBFT Pre-Prepare Phase



Caption: PBFT Prepare Phase



Caption: PBFT Commit Phase



6. Cryptographic Message Authentication

To prevent message spoofing and forgery, RSA-based digital signatures are used.

Each node generates:

- Private Key
- Public Key

Signing Process

1. Sender signs the message using its private key.
2. Receiver verifies the signature using the public key.
3. Invalid signatures are rejected.

Key Distribution

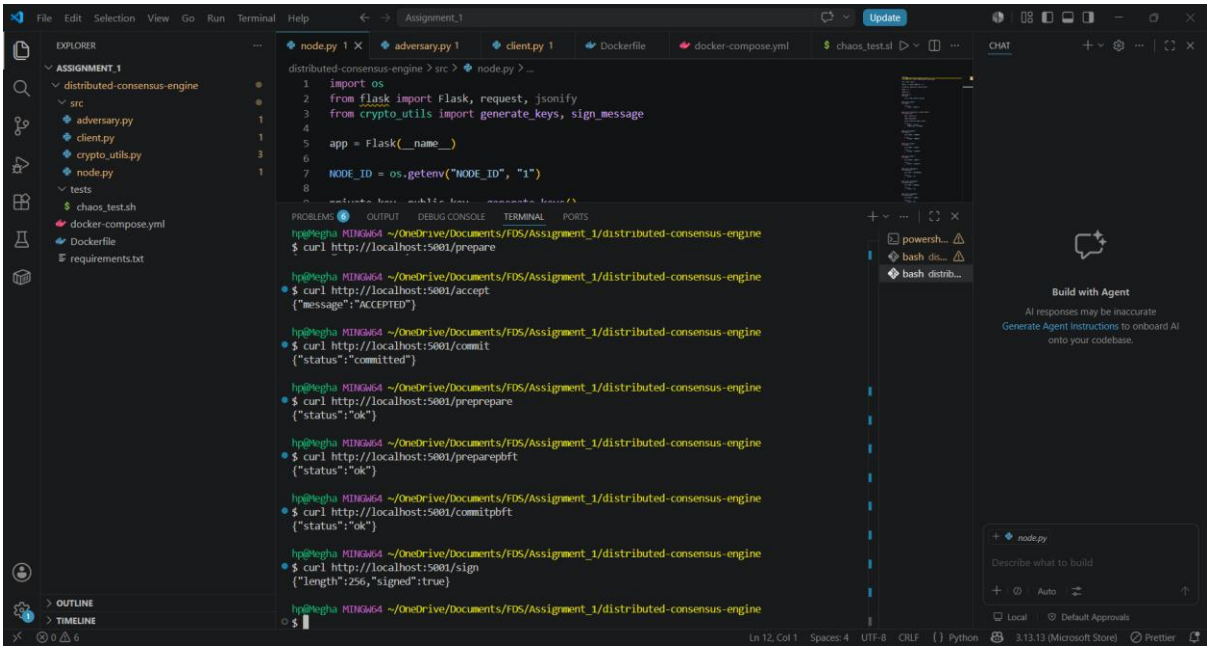
Public keys are shared among all nodes during initialization.

Private keys remain local to each node.

Benefits

- Authentication
- Integrity
- Non-repudiation

Caption: Successful Cryptographic Message Signing



7. Byzantine Adversary Node

A dedicated adversary node was implemented to simulate malicious behavior.

Attack Scenarios

1. Fake Transaction Injection
2. Message Forgery
3. Commit Suppression

Example:

Instead of sending:

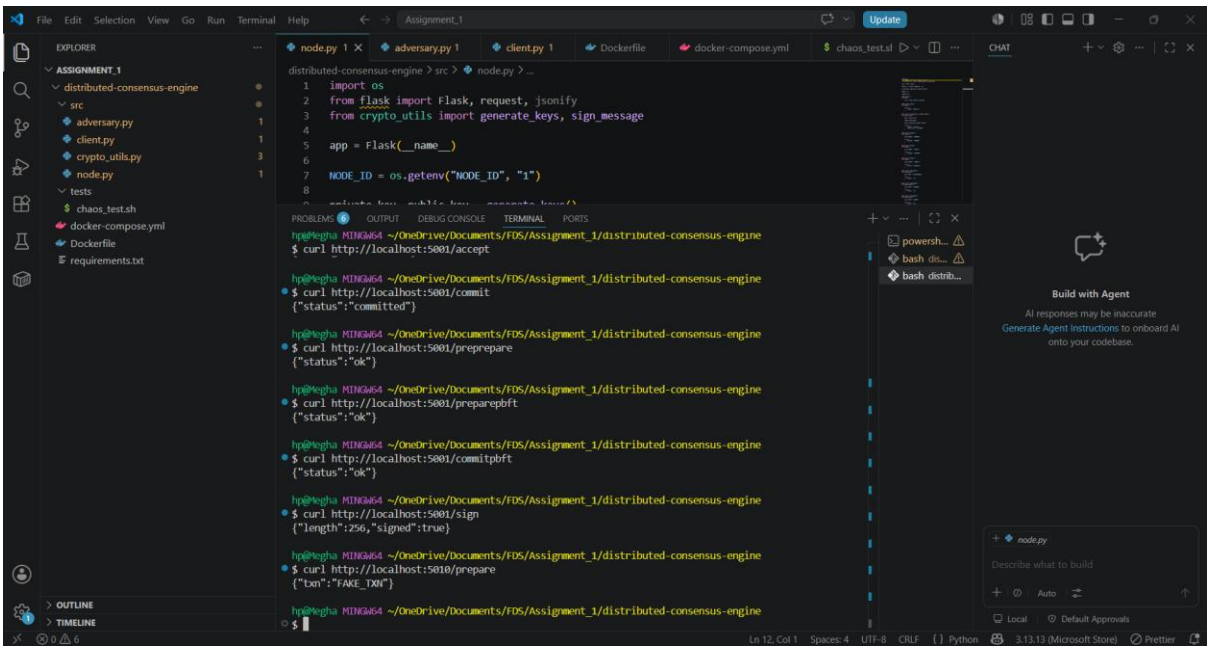
Transaction = TXN001

The adversary sends:

Transaction = FAKE_TXN

PBFT allows honest nodes to identify and ignore malicious activity.

Caption: Adversary Node Sending Fake Transaction



8. Docker-Based Deployment

Docker Compose is used to orchestrate all services.

Containers:

- node1
- node2
- node3
- node4
- node5
- adversary
- client

Advantages:

- Isolation
- Portability
- Reproducibility
- Easy fault injection

The entire cluster can be started using:

docker compose up –build

---

9. Chaos Testing and Failure Recovery

During chaos testing, the current leader node was intentionally stopped. The system successfully reassigned leadership to another active node, ensuring that transaction processing and consensus operations continued without complete service interruption. This demonstrates fault recovery and leader failover capability.

To evaluate resiliency, a chaos testing script was developed.

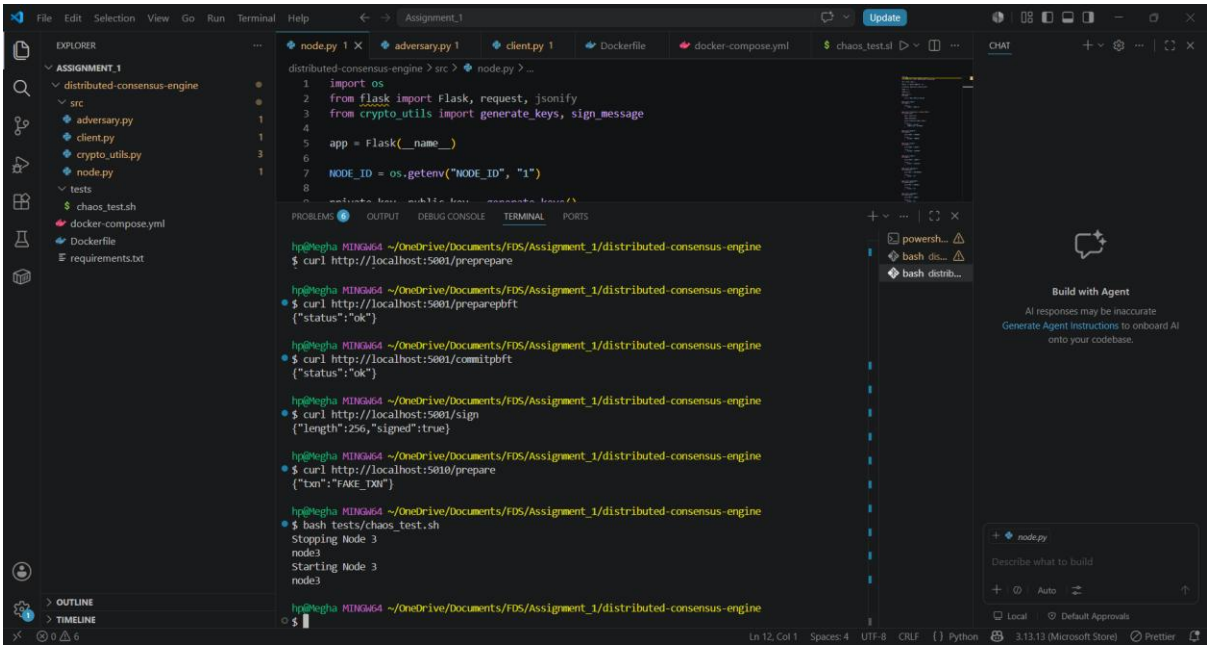
Test Scenario

1. Node3 is intentionally stopped.
2. Remaining nodes continue operation.
3. Transactions continue to be processed.
4. Node3 is restarted.
5. Cluster returns to normal operation.

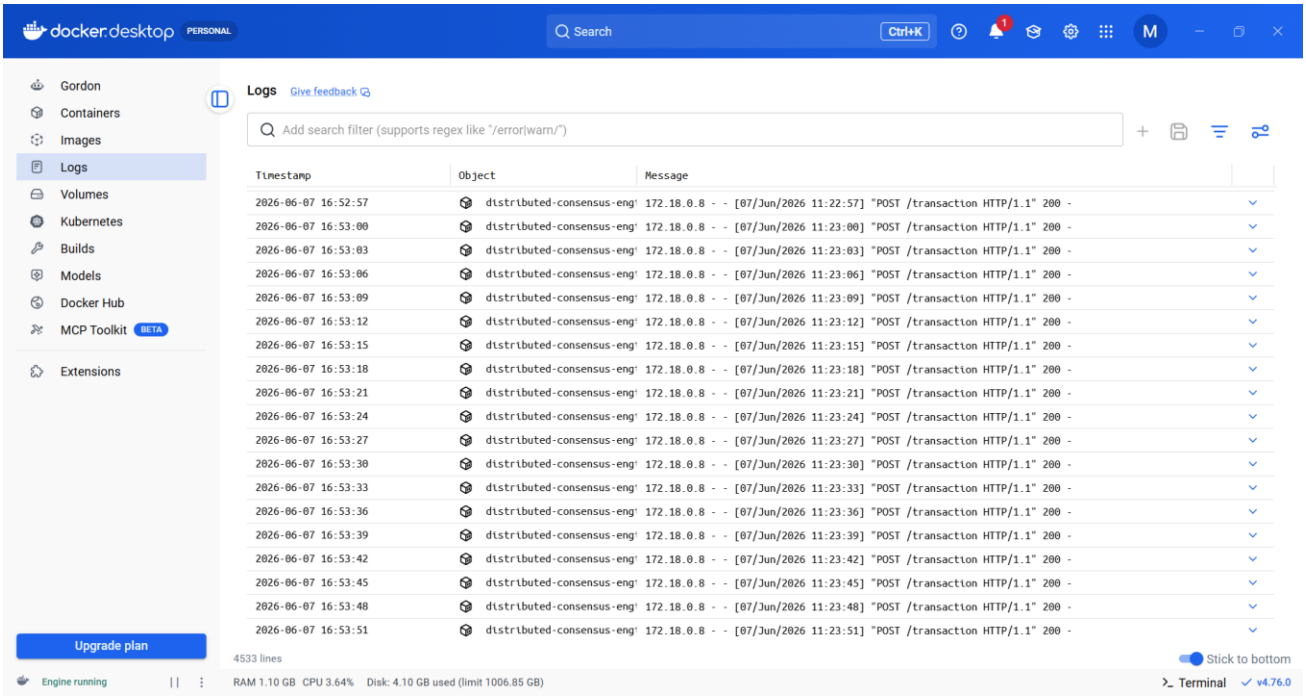
Results

The distributed system remained available during node failure and successfully recovered after restart.

Caption: Node3 Stopped During Chaos Test and Restarted Successfully



Caption: Continuous Transaction Processing During Failure



---

10. Experimental Observations

Observed Characteristics:

Feature	Observation
Leader Election	Successfully reassigned leadership after failure simulation
Paxos	Achieved majority-based agreement
PBFT	Detected Byzantine behavior
Cryptography	Verified message authenticity
Docker Deployment	Stable container execution
Chaos Testing	Recovered from node failures

The implementation successfully demonstrated crash fault tolerance and Byzantine fault tolerance in a containerized distributed environment.

---

11. Video Demonstration Link

Video Link:

https://drive.google.com/file/d/18sK3tvSHSY4BE5O3D_BiXMfhh2Y_hgRQ/view?usp=sharing

The video demonstrates:

1. Docker Compose Build

2. Normal Transaction Processing
3. Paxos Execution
4. PBFT Execution
5. Byzantine Adversary Demonstration
6. Chaos Testing and Recovery

---

12. Conclusion

This project implemented a resilient distributed state machine replication system capable of handling both crash faults and Byzantine faults.

Leader election provided stable coordination among nodes, Paxos ensured consensus during crash failures, and PBFT protected the system against malicious participants. Cryptographic signatures enhanced message authenticity, while Docker based deployment enabled efficient orchestration and testing.

The system successfully demonstrated reliable transaction processing, fault tolerance, and recovery capabilities in a distributed environment.